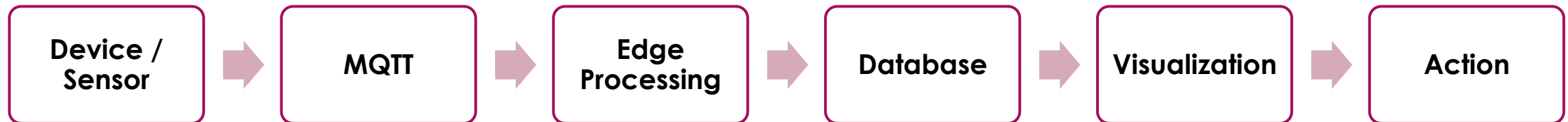


IoT Building Blocks – Hands-on Lab

IoT Pipeline – Hands on

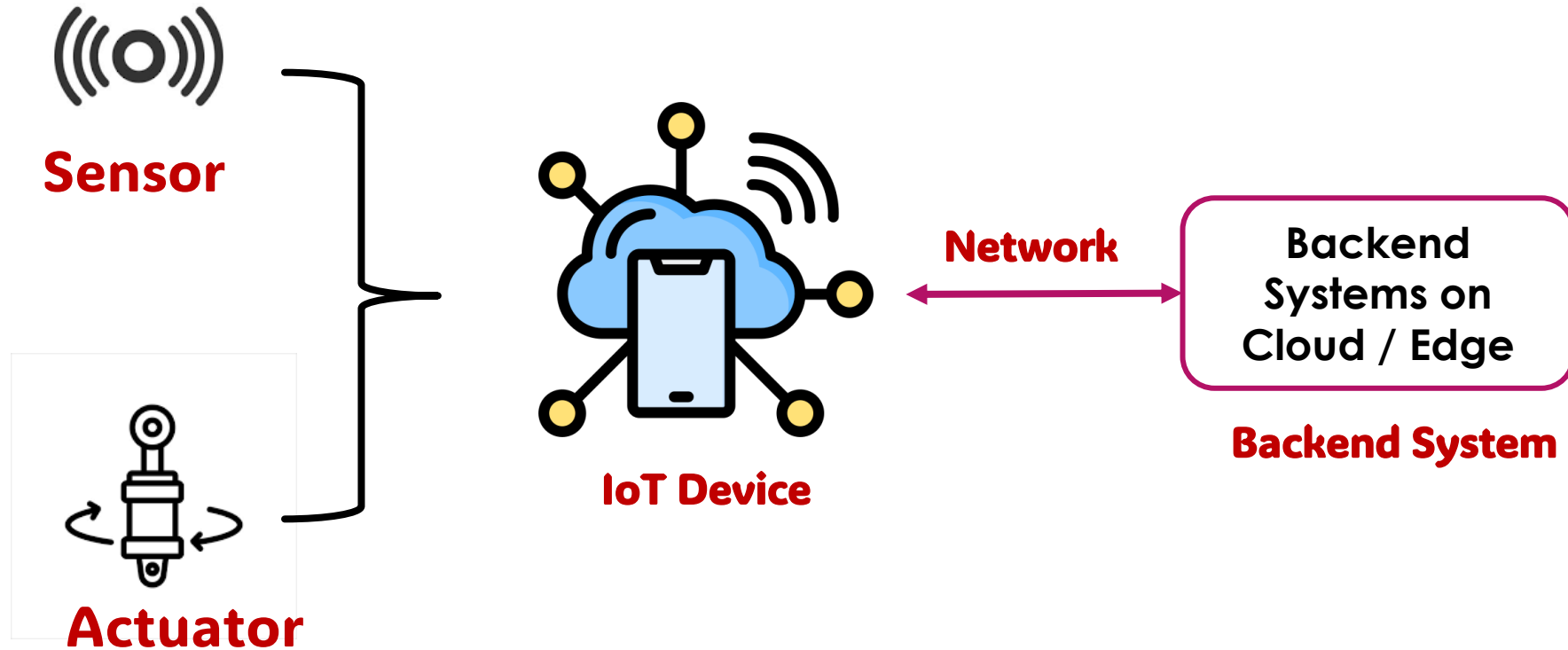
IoT Building Blocks – Hands-on Lab

IoT Pipeline – Hands on

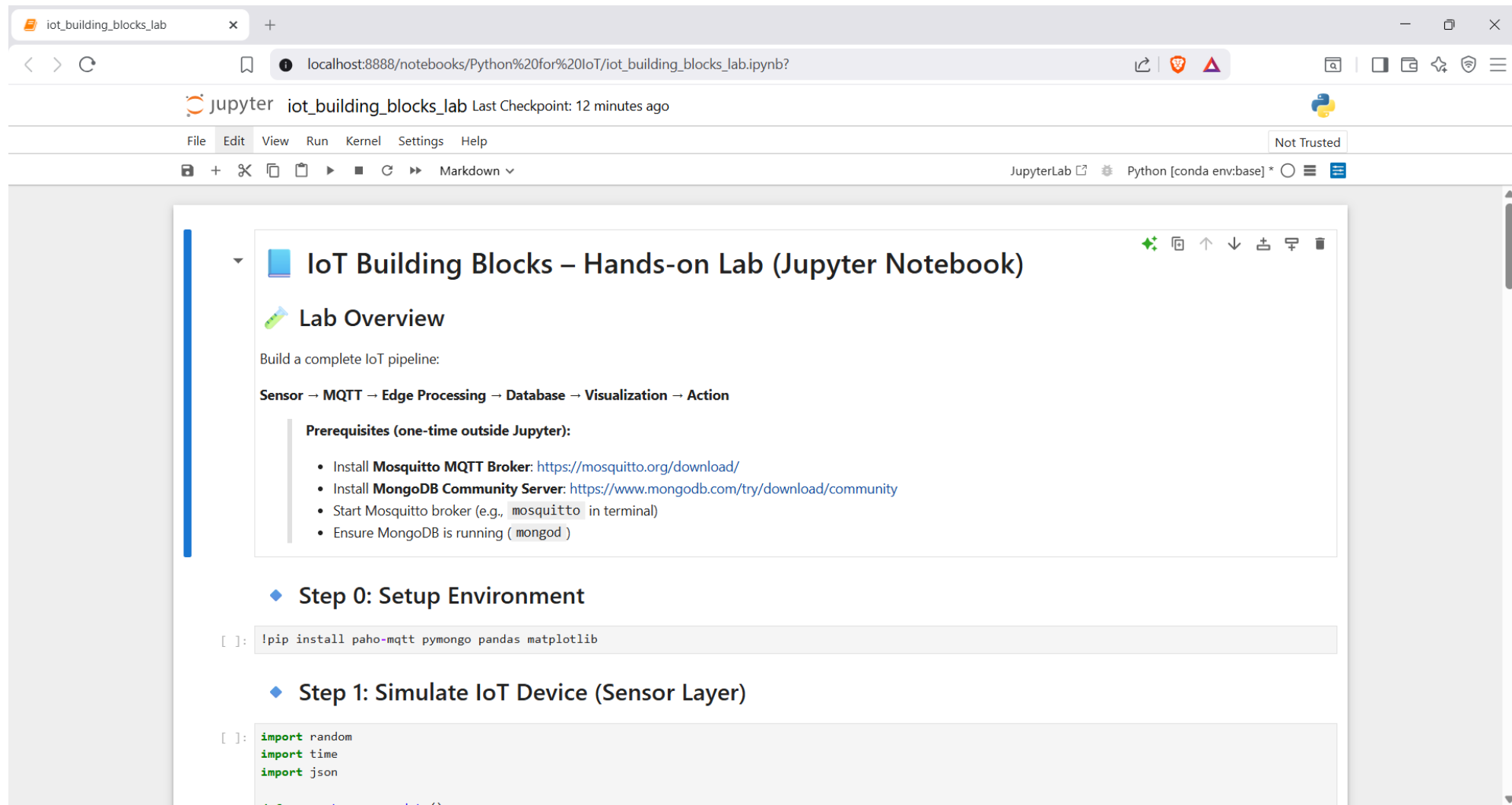


IoT Building Blocks – Hands-on Lab

Device Simulator



IoT Building Blocks – Environment Setup



The screenshot shows a JupyterLab interface in a web browser. The browser address bar shows the URL: `localhost:8888/notebooks/Python%20for%20IoT/iot_building_blocks_lab.ipynb?`. The JupyterLab header shows the notebook title "iot_building_blocks_lab" and a "Last Checkpoint: 12 minutes ago" message. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main content area displays the notebook content, which includes a title "IoT Building Blocks – Hands-on Lab (Jupyter Notebook)", a "Lab Overview" section, and two code cells.

IoT Building Blocks – Hands-on Lab (Jupyter Notebook)

Lab Overview

Build a complete IoT pipeline:

Sensor → MQTT → Edge Processing → Database → Visualization → Action

Prerequisites (one-time outside Jupyter):

- Install **Mosquitto MQTT Broker**: <https://mosquitto.org/download/>
- Install **MongoDB Community Server**: <https://www.mongodb.com/try/download/community>
- Start Mosquitto broker (e.g., `mosquitto` in terminal)
- Ensure MongoDB is running (`mongod`)

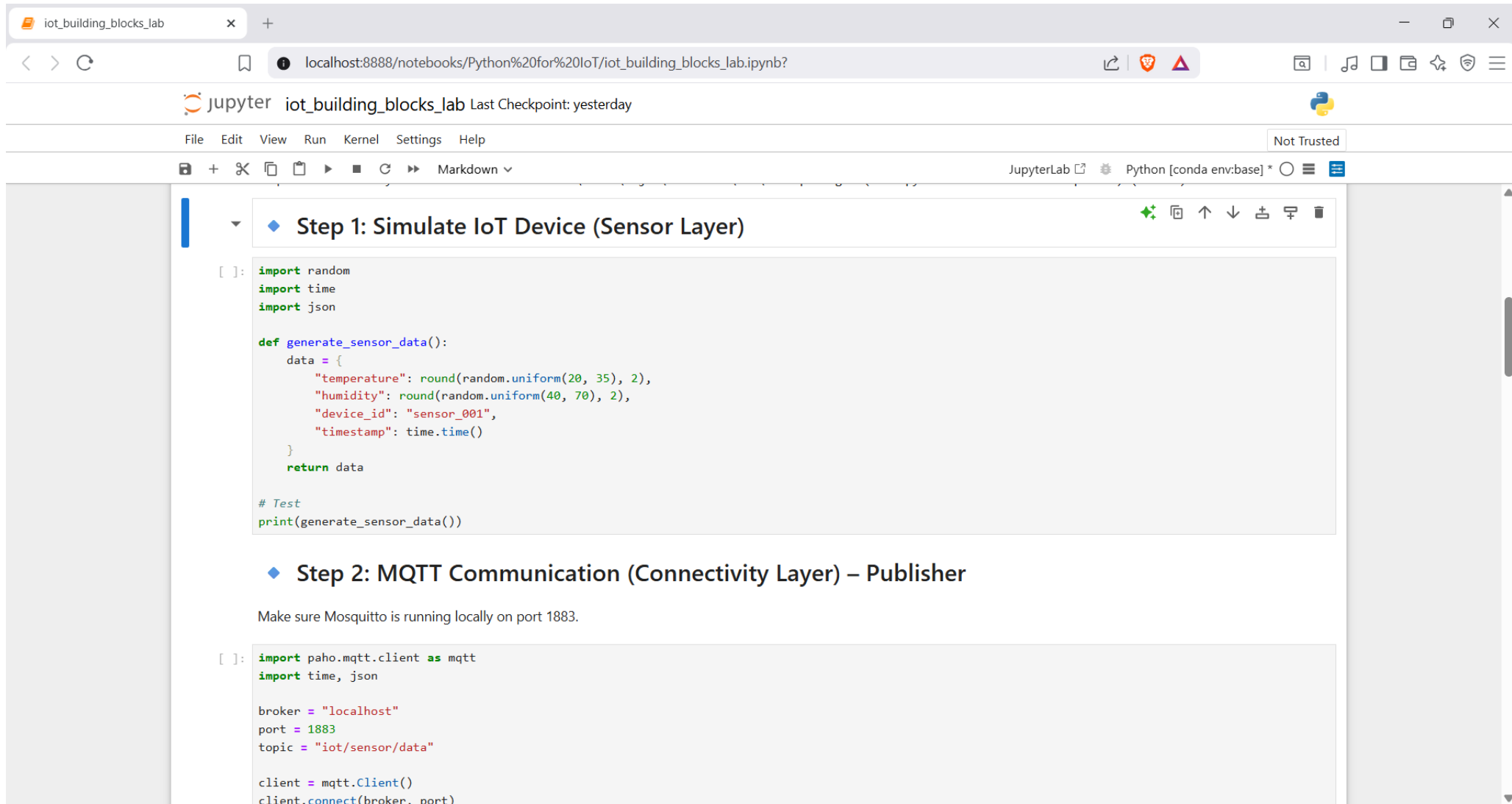
Step 0: Setup Environment

```
[ ]: !pip install paho-mqtt pymongo pandas matplotlib
```

Step 1: Simulate IoT Device (Sensor Layer)

```
[ ]: import random
import time
import json
```

IoT Building Blocks Virtual IOT Device



The screenshot displays a JupyterLab environment with a browser window at the top showing the URL `localhost:8888/notebooks/Python%20for%20IoT/iot_building_blocks_lab.ipynb`. The notebook interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main content area shows a Python notebook with two sections:

Step 1: Simulate IoT Device (Sensor Layer)

```
[ ]: import random
import time
import json

def generate_sensor_data():
    data = {
        "temperature": round(random.uniform(20, 35), 2),
        "humidity": round(random.uniform(40, 70), 2),
        "device_id": "sensor_001",
        "timestamp": time.time()
    }
    return data

# Test
print(generate_sensor_data())
```

Step 2: MQTT Communication (Connectivity Layer) – Publisher

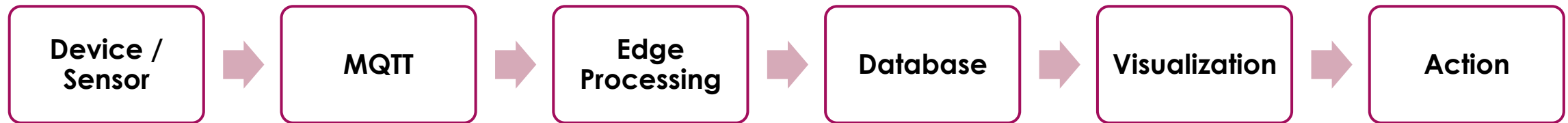
Make sure Mosquitto is running locally on port 1883.

```
[ ]: import paho.mqtt.client as mqtt
import time, json

broker = "localhost"
port = 1883
topic = "iot/sensor/data"

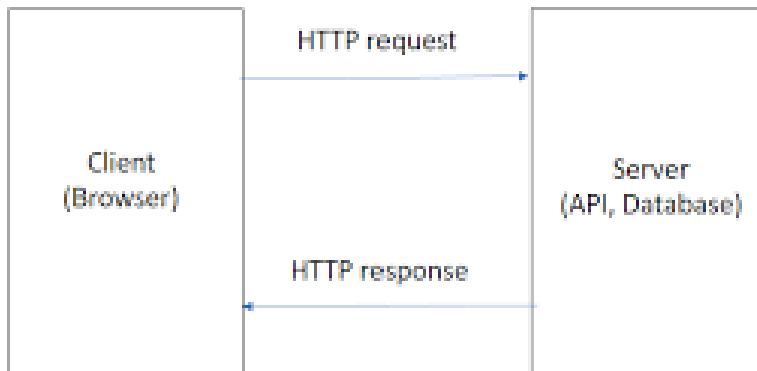
client = mqtt.Client()
client.connect(broker, port)
```

IoT Building Blocks - IOT Device – MQTT Communication



Communication models

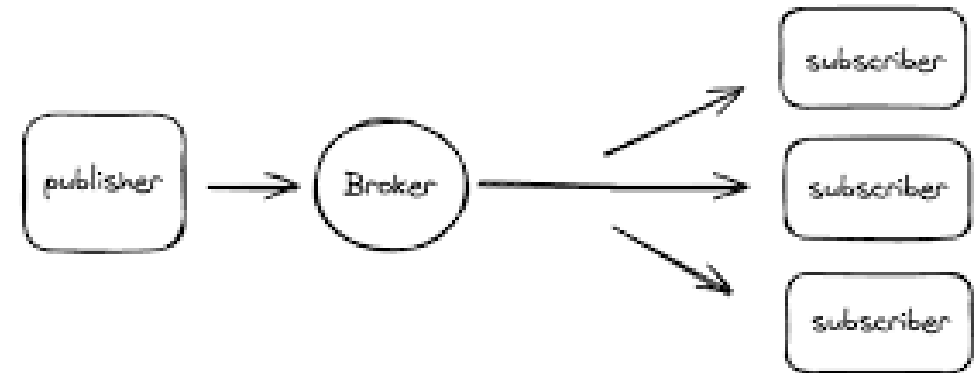
Request Response (Client/Server) communication model



Popular Request / response protocols in IoT

- HTTP/REST
- gRPC
- SOAP

Publisher Subscriber (Pub-Sub) communication model



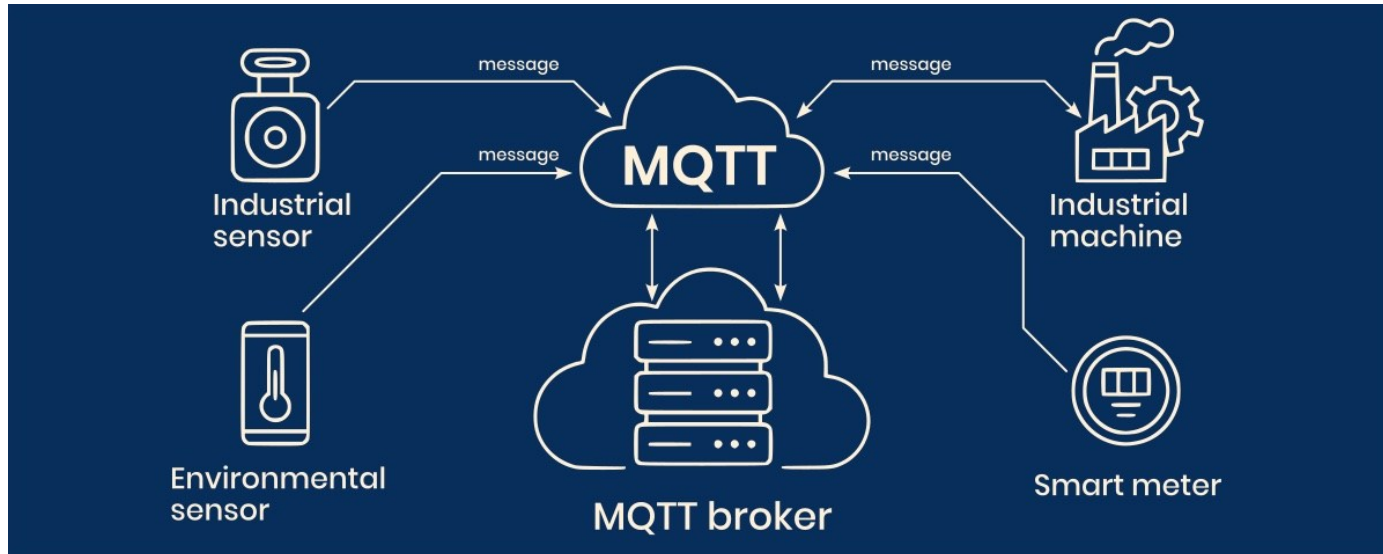
Popular Request / response protocols in IoT

- MQTT
- AMQP
- NATS

What is MQTT ?

MQTT - Message Queuing Telemetry Transport

- Lightweight pub/sub messaging protocol. Designed for IoT & M2M communication.
- Widely used in
 - **IoT & IIoT:** Connecting sensors & embedded devices.
 - **Home Automation:** Interconnecting smart systems.
 - **Remote Monitoring:** Oil, gas, and utility monitoring



Key Components

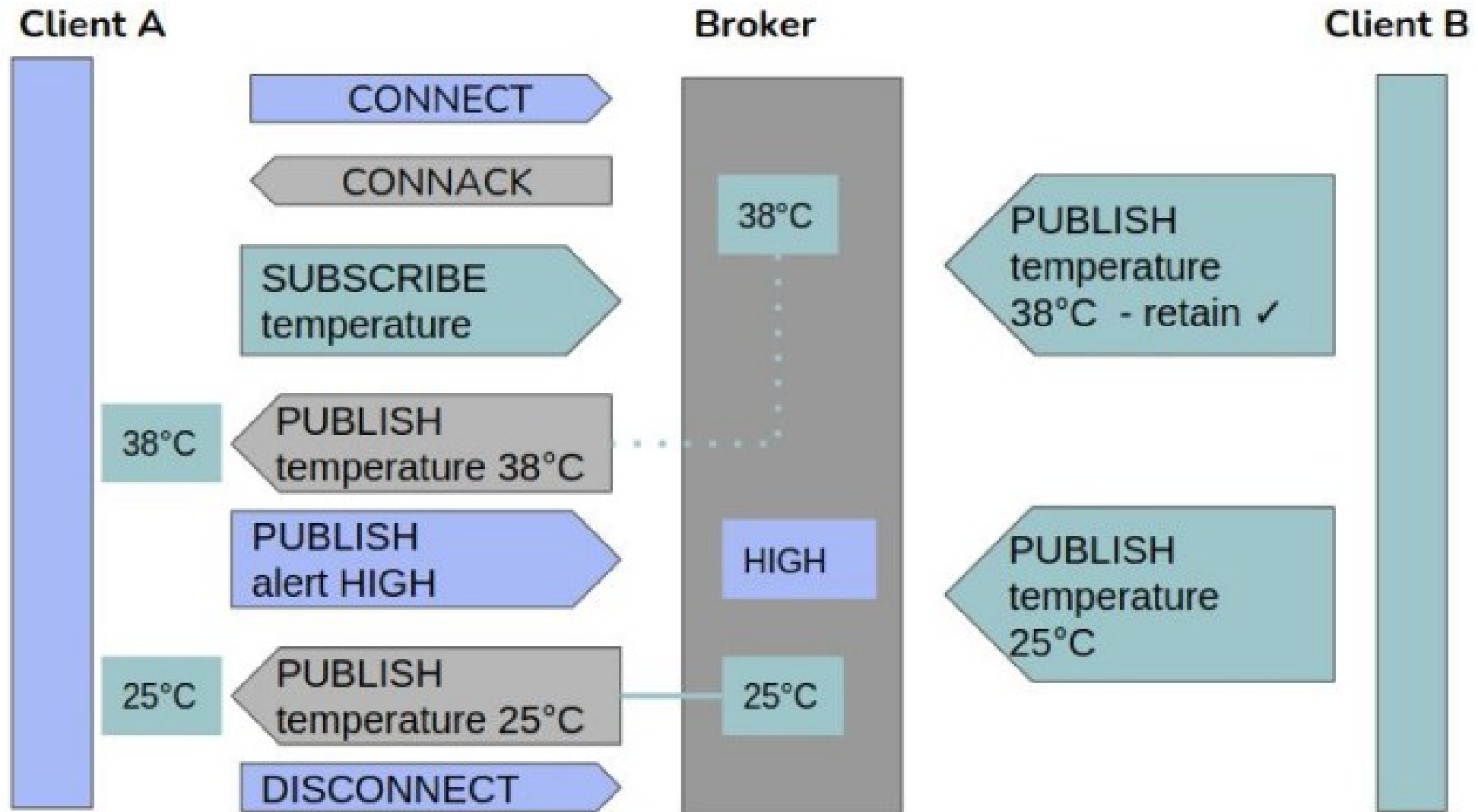
Broker: Acts as the central hub, receiving messages from publishers and distributing them to subscribers.

Publisher: Devices/sensors that send data to a specific topic.

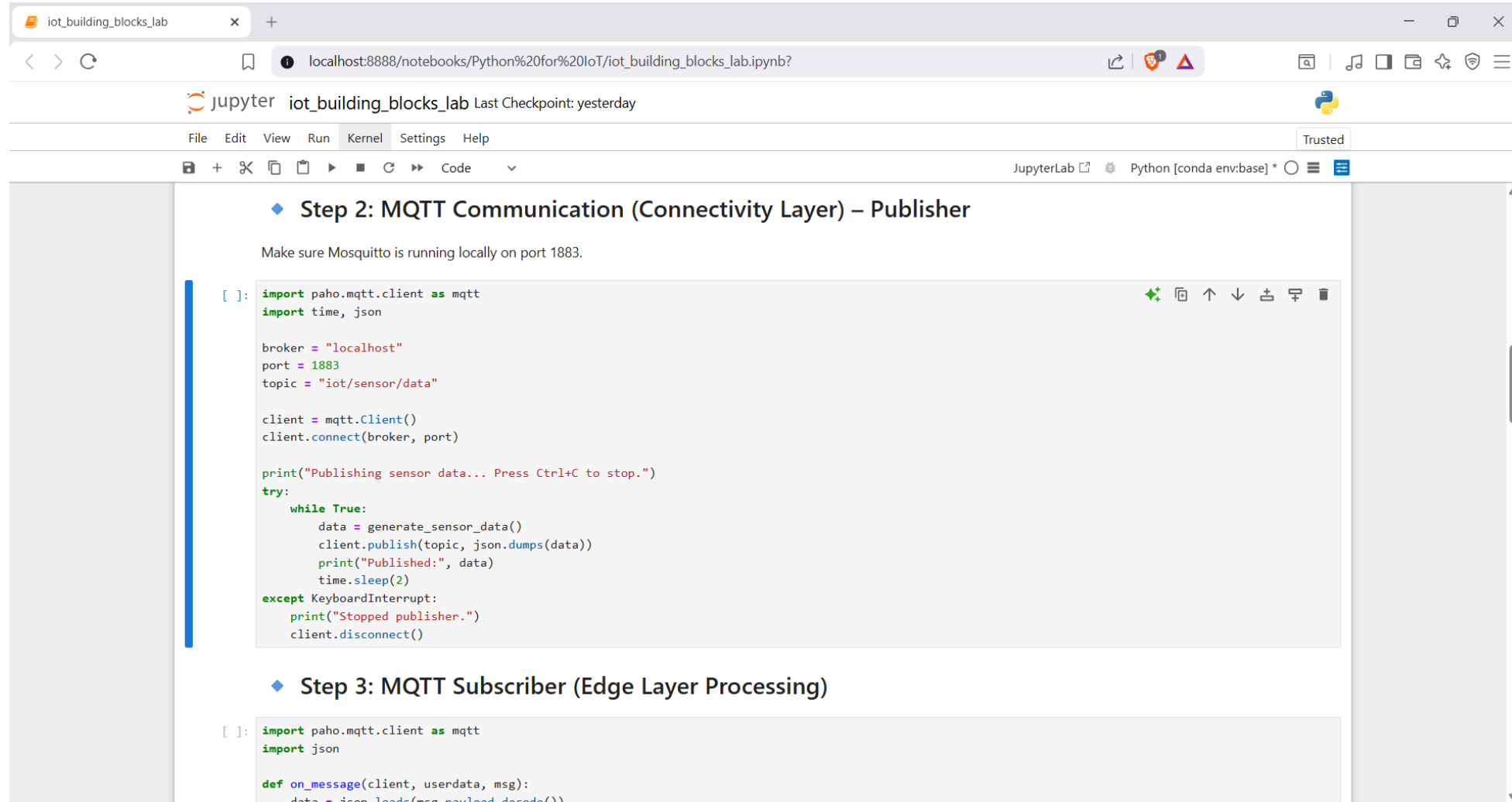
Subscriber: Devices that express interest (subscribe) to specific topics to receive data.

Topic: The mechanism for routing messages.

How MQTT Works ?



IOT Device – MQTT Communication



The screenshot shows a JupyterLab window titled 'iot_building_blocks_lab'. The browser address bar shows 'localhost:8888/notebooks/Python%20for%20IoT/iot_building_blocks_lab.ipynb?'. The JupyterLab interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main area displays a notebook with two sections:

◆ Step 2: MQTT Communication (Connectivity Layer) – Publisher

Make sure Mosquitto is running locally on port 1883.

```
[ ]: import paho.mqtt.client as mqtt
import time, json

broker = "localhost"
port = 1883
topic = "iot/sensor/data"

client = mqtt.Client()
client.connect(broker, port)

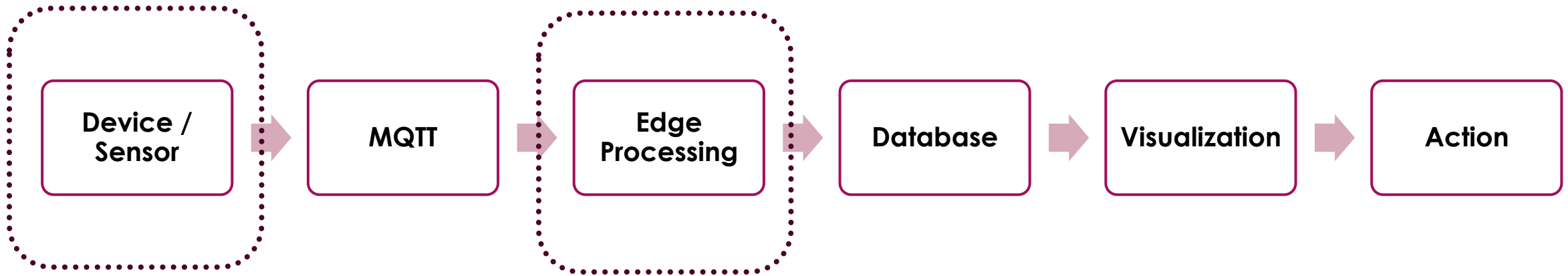
print("Publishing sensor data... Press Ctrl+C to stop.")
try:
    while True:
        data = generate_sensor_data()
        client.publish(topic, json.dumps(data))
        print("Published:", data)
        time.sleep(2)
except KeyboardInterrupt:
    print("Stopped publisher.")
    client.disconnect()
```

◆ Step 3: MQTT Subscriber (Edge Layer Processing)

```
[ ]: import paho.mqtt.client as mqtt
import json

def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
```

IOT Device – MQTT Communication Subscriber (Edge)



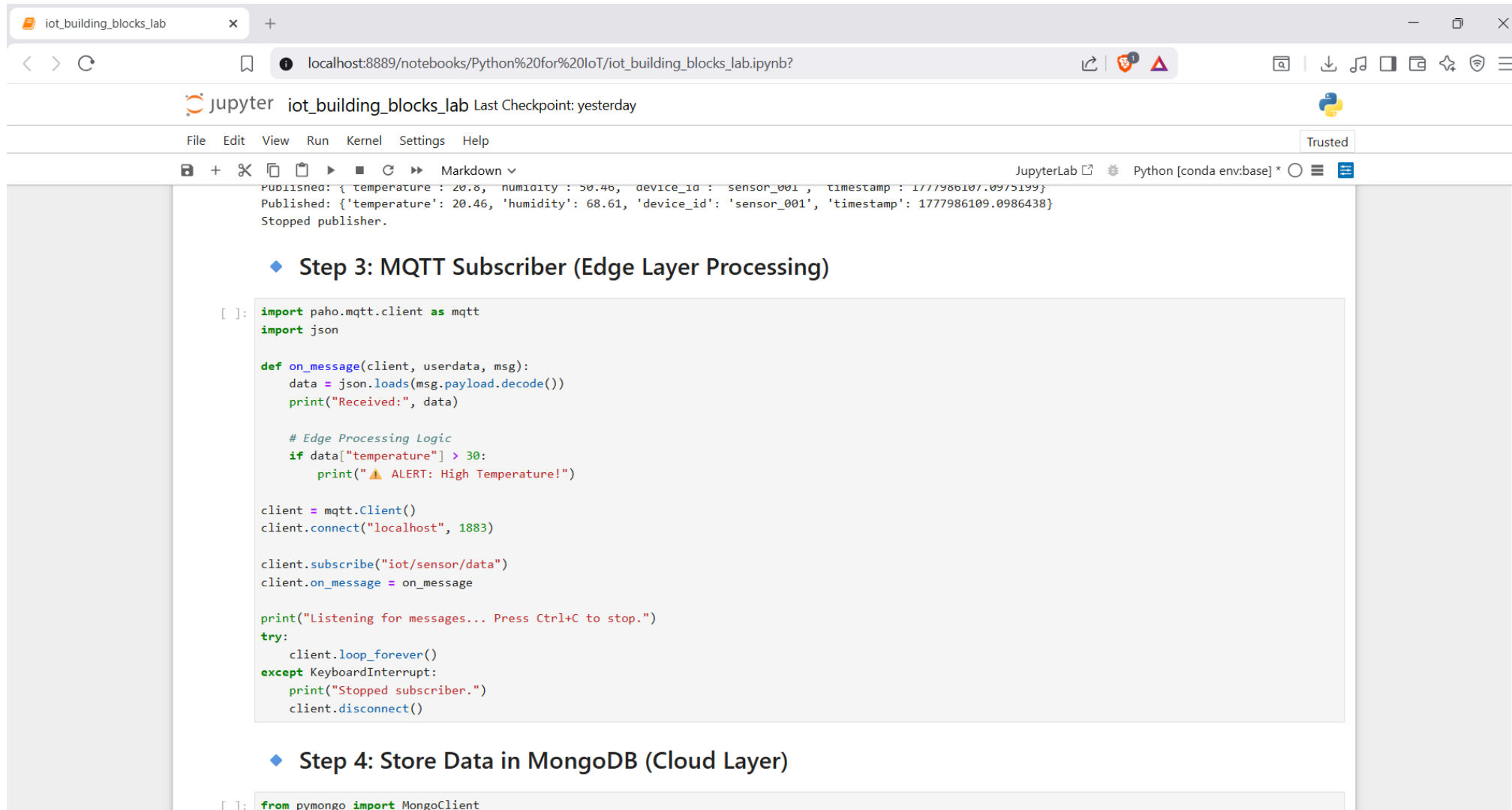
Publisher

- Publishes to a topic

Subscriber

- subscribes to a specific topic
- and whenever new data is published to that topic...it receives it instantly.

IOT Device – MQTT Communication Subscriber (Edge)



The screenshot shows a JupyterLab interface with a notebook titled 'iot_building_blocks_lab'. The browser address bar shows 'localhost:8889/notebooks/Python%20for%20IoT/iot_building_blocks_lab.ipynb?'. The notebook content includes a previous cell with MQTT publisher logs and a new cell titled 'Step 3: MQTT Subscriber (Edge Layer Processing)'. The code in the new cell imports paho.mqtt.client and json, defines an on_message callback, and sets up a client to subscribe to 'iot/sensor/data' on localhost:1883. It also includes a try-except block for KeyboardInterrupt to stop the subscriber. The bottom of the image shows the start of 'Step 4: Store Data in MongoDB (Cloud Layer)' with the import 'from pymongo import MongoClient'.

```
Published: { 'temperature': 20.8, 'humidity': 50.46, 'device_id': 'sensor_001', 'timestamp': 1777986107.0975199 }
Published: { 'temperature': 20.46, 'humidity': 68.61, 'device_id': 'sensor_001', 'timestamp': 1777986109.0986438 }
Stopped publisher.

♦ Step 3: MQTT Subscriber (Edge Layer Processing)

[ ]: import paho.mqtt.client as mqtt
import json

def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    print("Received:", data)

    # Edge Processing Logic
    if data["temperature"] > 30:
        print("⚠️ ALERT: High Temperature!")

client = mqtt.Client()
client.connect("localhost", 1883)

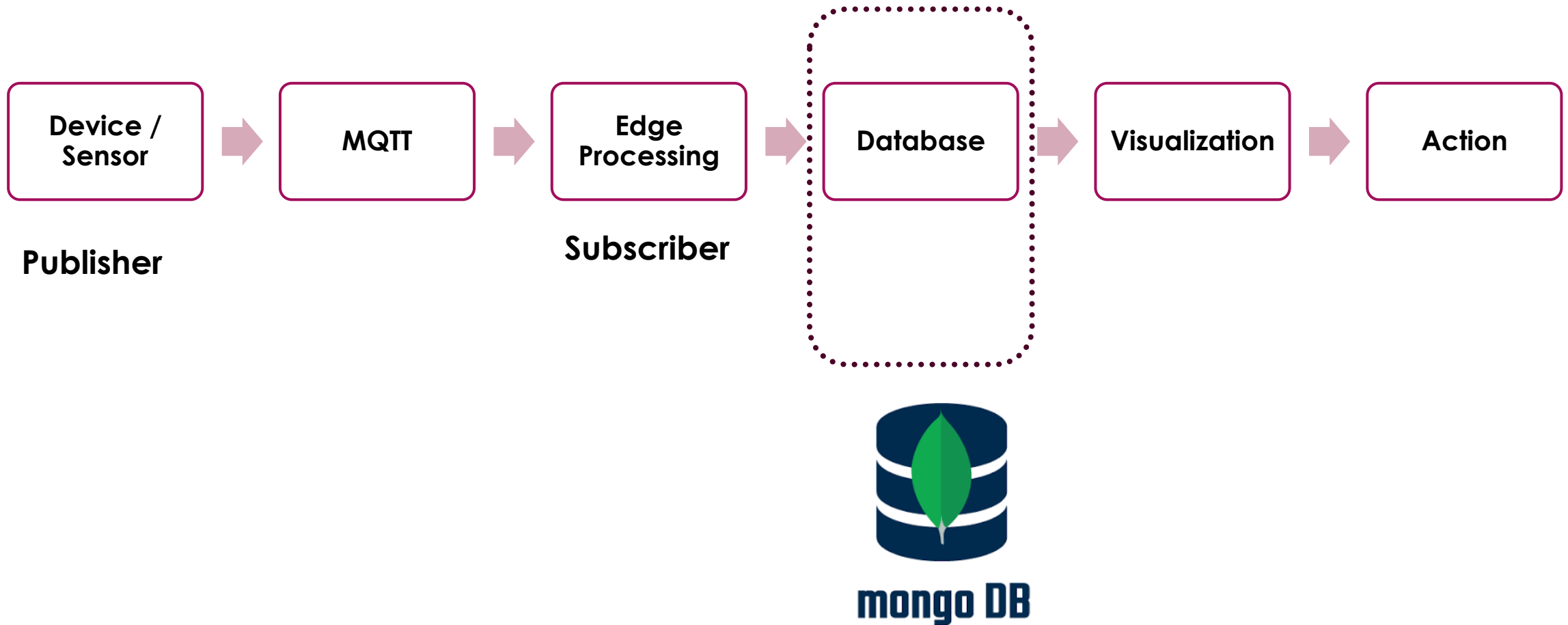
client.subscribe("iot/sensor/data")
client.on_message = on_message

print("Listening for messages... Press Ctrl+C to stop.")
try:
    client.loop_forever()
except KeyboardInterrupt:
    print("Stopped subscriber.")
    client.disconnect()

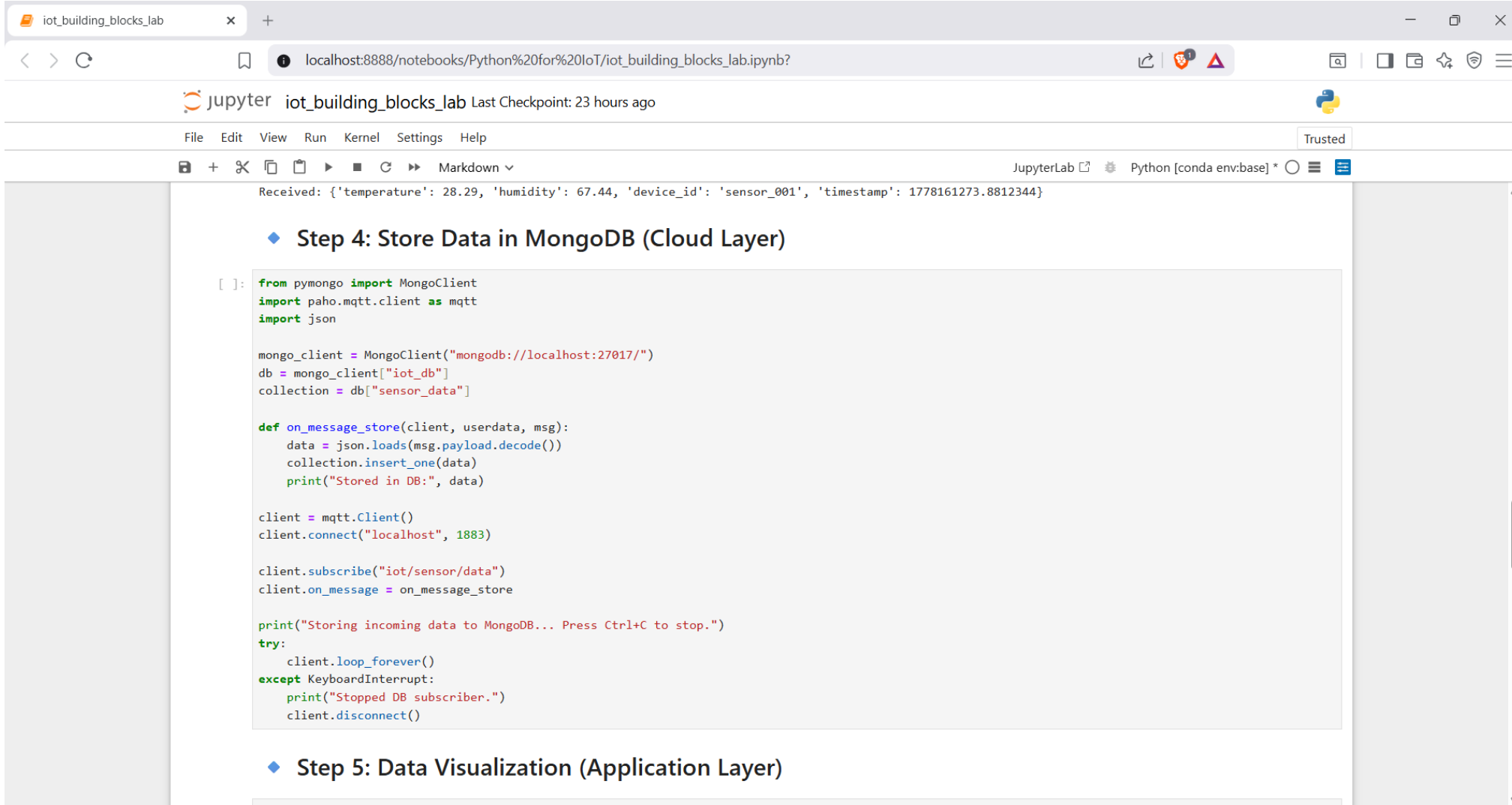
♦ Step 4: Store Data in MongoDB (Cloud Layer)

[ ]: from pymongo import MongoClient
```

IOT Device – Store Data in MongoDB (Cloud Layer)



IOT Device – Store Data in MongoDB (Cloud Layer)



The screenshot shows a JupyterLab interface with a browser window at the top displaying the URL `localhost:8888/notebooks/Python%20for%20IoT/iot_building_blocks_lab.ipynb?`. The notebook title is `iot_building_blocks_lab` and it shows the last checkpoint was 23 hours ago. The menu bar includes File, Edit, View, Run, Kernel, Settings, and Help. The toolbar shows various icons for file operations and a 'Trusted' status indicator. The main area displays a code cell with the following Python code:

```
Received: {'temperature': 28.29, 'humidity': 67.44, 'device_id': 'sensor_001', 'timestamp': 1778161273.8812344}

◆ Step 4: Store Data in MongoDB (Cloud Layer)

[ ]: from pymongo import MongoClient
import paho.mqtt.client as mqtt
import json

mongo_client = MongoClient("mongodb://localhost:27017/")
db = mongo_client["iot_db"]
collection = db["sensor_data"]

def on_message_store(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    collection.insert_one(data)
    print("Stored in DB:", data)

client = mqtt.Client()
client.connect("localhost", 1883)

client.subscribe("iot/sensor/data")
client.on_message = on_message_store

print("Storing incoming data to MongoDB... Press Ctrl+C to stop.")
try:
    client.loop_forever()
except KeyboardInterrupt:
    print("Stopped DB subscriber.")
    client.disconnect()
```

Below the code cell, the next step is indicated:

◆ Step 5: Data Visualization (Application Layer)